

# 实验3：页表与内存管理

## 框架文件速查

| 本实验任务       | 对应文件                    | TODO标注          |
|-------------|-------------------------|-----------------|
| 任务1：物理页分配器  | kernel/mm/kalloc.c      | TODO [Lab3-任务1] |
| 任务2：三级页表查找  | kernel/mm/vm.c          | TODO [Lab3-任务2] |
| 任务3：批量建立映射  | kernel/mm/vm.c          | TODO [Lab3-任务3] |
| 任务4：开启MMU分页 | kernel/mm/vm.c          | TODO [Lab3-任务4] |
| 任务5：加入构建    | Makefile<br>(SRCS 添加两行) | —               |

## 实验目标

在实验1和2中，我们开启了机器，并可以在”物理内存”上随心所欲地读写。但在现代操作系统中，让程序直接操控物理内存是极其危险的！本实验旨在带领大家实现 RISC-V 的 Sv39 分页机制。你将亲手写出一个物理内存分配器（维护空闲内存链表），并构建三级页表结构，最终开启 `satp` 寄存器，让 CPU 迈入”虚拟内存”的奇妙世界。

## 理论补充：虚拟内存的”楚门世界”

在没有虚拟内存时，CPU 发出的地址直接对应物理内存条上的位置。这面临两个难题：

1. **不安全**：任何一个越界指针都可能把操作系统核心数据给覆盖掉。
2. **碎片化**：如果要给应用分配 4MB 连续内存，但物理内存只有零散的可用块，分配就会失败。

引入页表（Page Table）后，CPU 开启了”翻译机”。程序中使用的地址变成了**虚拟地址**（Virtual Address, VA）。MMU 硬件会查页表，将 VA 翻译为真正的**物理地址**（Physical Address, PA）。

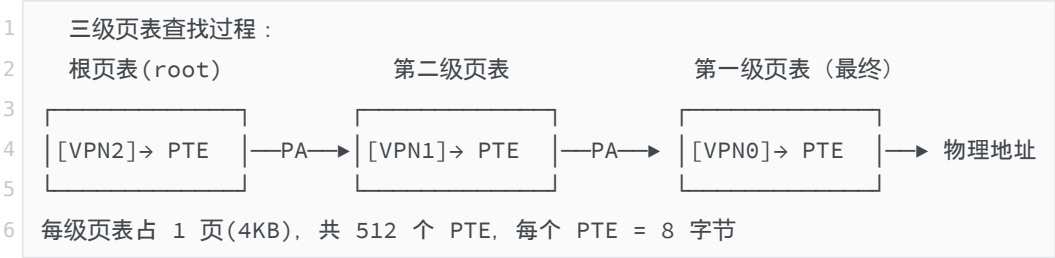
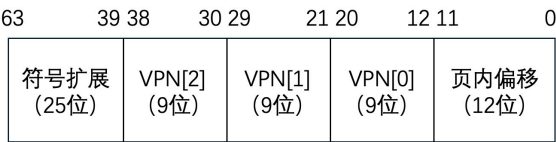
在 RISC-V 的 Sv39 标准中：

- 内存被划分为一页一页的，每页大小为 4KB (4096 字节)。
- 页表是三级树状结构，每次寻址需要查三次表（对应 `VPN[2]`，`VPN[1]`，`VPN[0]`）。
- 每个页表项（PTE）除了物理页号外，还有权限位（V:有效, R:可读, W:可写, X:可执行, U:用户可访问）。

本实验的任务，就是要亲手实现这套物理内存页面分配与页表查表机制。

### ★ Sv39 虚拟地址结构图

Sv39 虚拟地址（39位有效）：



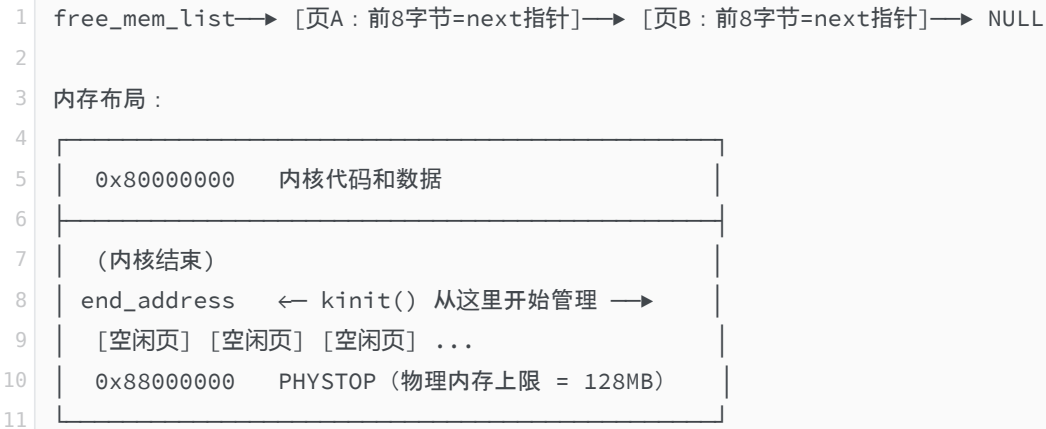
关键宏（已在 `kernel/include/riscv.h` 中提供，使用前先阅读定义）：

| 宏                          | 功能                                |
|----------------------------|-----------------------------------|
| <code>PX(level, va)</code> | 从虚拟地址 va 中提取第 level 级的 9 位页表索引    |
| <code>PA2PTE(pa)</code>    | 物理地址 → PTE 格式                     |
| <code>PTE2PA(pte)</code>   | PTE → 物理地址（反向操作）                  |
| <code>PTE_V</code>         | 有效位标志 (0x1)                       |
| <code>MAKE_SATP(pt)</code> | 生成写入 satp 寄存器的值 (Sv39模式 + 根页表PPN) |

## 任务指南与实施步骤

### 任务1：实现物理内存的”回收站”（物理页面分配器）

原理：xv6 采用了一种极其巧妙的数据结构，将空闲的物理页本身的前 8 字节存放一个 `next` 指针，把所有空闲页面串成一个单向链表 `free_mem_list`（具体数据结构为 `struct free_node`）。



目标文件：`kernel/mm/kalloc.c`，填写三处 TODO：

`kinit()`：遍历从内核结束 (`end_address`) 到 `PHYSTOP` 之间的每一个物理页，将其加入 `free_mem_list`。

- 思考：起始地址需要按 4KB 对齐（使用 `PGROUNDUP`），每次步进 `PGSIZE`。

`kfree(pa)`：将物理页 `pa` 插入 `free_mem_list`。

- 思考：如何把一个物理基地址强转为 `struct free_node*` 指针并“伪装”成链表节点？如何完成插入（只需3步：设next、更新head）？
- 安全检查：地址必须按 4KB 对齐（`pa % PGSIZE == 0`）且在合法范围内（大于等于 `end_address`，小于 `PHYSTOP`）。

`kalloc()`：从 `free_mem_list` 摘除头节点，返回该页的地址。

- 思考：如果 `free_mem_list == 0`，说明内存耗尽，该返回什么？
- 

## 任务2：实现虚拟地址到物理地址的导航（页表遍历 `walk`）

原理：`walk` 沿三级页表从根向叶逐级“走下去”。对于 `level=2` 和 `level=1` 两级，每次需要：

- 用 `PX(level, va)` 提取当前级的索引，定位到 PTE。
- 若该 PTE 有效（`PTE_V` 位为1）→ 用 `PTE2PA` 提取下一级页表的基地址。
- 若该 PTE 无效且 `alloc = 1` → 调 `kalloc()` 分配新页，清零（必须！否则脏数据导致寻址混乱），然后将新页表地址用 `PA2PTE | PTE_V` 写入当前 PTE。
- 若无效且 `alloc = 0` → 直接返回 0。

关键问题：为什么新分配的页表必须清零？若不清零，`PTE_V` 位随机为1会发生什么？

目标文件：`kernel/mm/vm.c`，填写三处 `TOD0 [Lab3-任务2]`。

## 任务3：搭建虚拟机的“楚门世界”（建立映射 `mappages`）

原理：对于虚拟地址区间 `[va, va+size)` 中的每一个页，调用 `walk` 找到对应的叶节点 PTE，然后将物理地址和权限写入该 PTE。

一个 PTE 的完整格式：

```
1 [63:10] = PPN (物理页号, 来自 PA2PTE(pa))
2 [9:0]   = 权限位 (perm | PTE_V)
```

关键问题：

- 框架中的 `mappages` 已经写好了 `for` 循环推进物理/虚拟地址并检查重复映射的逻辑。你需要做的仅仅是顺水推舟填入映射项。

目标文件：`kernel/mm/vm.c`，填写 `TOD0 [Lab3-任务3]`（将传入的物理地址 `pa` 结合权限位并写入 `*pte`）。

## 任务4：建立内核页表与开启 MMU

原理：`kvmininit()` 调用 `mappages` 建立内核的虚拟→物理地址映射，`kvminithart()` 将内核根页表地址写入 `satp` 寄存器触发 MMU。

内核使用恒等映射（虚拟地址 = 物理地址），这样切换到虚拟内存后，内核代码的指针值无需改变仍然有效。需要映射的区域：

| 虚拟地址范围            | 对应物理范围                | 权限  | 用途        |
|-------------------|-----------------------|-----|-----------|
| UART0             | UART0<br>(0x10000000) | R+W | 串口设备      |
| [KERNBASE, etext) | 同上                    | R+X | 内核代码段     |
| [etext, PHYSTOP)  | 同上                    | R+W | 内核数据+空闲内存 |

`kvminithart()` 关键点:

1. 调用 `w_satp(MAKE_SATP(kernel_pagetable))` 写入根页表并开启 Sv39 模式。
2. 立即调用 `sfence_vma()` 刷新 TLB, 否则 CPU 可能用旧的 (无效的) 地址翻译继续执行。

关键问题: 如果 `kvminithart()` 执行后指令崩溃, 最可能的原因是什么? (提示: 检查内核代码段的映射是否包含了 `PTE_X`)

### ★ 任务5: 将 `kalloc.c` 和 `vm.c` 加入构建并更新 `main.c`

更新 `Makefile`, 在 `SRCS` 中追加:

```
1 kernel/mm/kalloc.c \
2 kernel/mm/vm.c
```

更新 `kernel/boot/main.c`, 在 `start_main()` 顶部添加初始化顺序 (顺序很重要, 不能颠倒):

```
1 void start_main() {
2     kinit();           // 1. 必须第一步: 建立 free_mem_list, 此后才能 kalloc
3     kvmininit();       // 2. 建立内核页表 (内部会 kalloc 页表页)
4     kvminithart();     // 3. 写 satp, 开启 MMU
5     clear_screen();
6     printf("Memory initialized. Paging enabled!\n");
7     while(1);
8 }
```

运行验收: `make run` 后输出 `Paging enabled!` 即通过。若系统直接崩溃, 优先检查 `kvmininit()` 的映射区域和权限。

### 避坑与故障排除策略 (Debugging)

1. 开启分页后立即崩溃: 说明 `kvmininit()` 的某段 `mappages` 映射有误。最常见是: 内核代码段忘了加 `PTE_X`, 或者 `UART0` 的地址写错了。用 GDB 在 `w_satp` 指令前设断点, 确认 `kernel_pagetable` 指向的内存不为全零。

2. `kfree` 调用后系统崩溃：检查传入的地址是否真正 4KB 对齐，以及是否在 `[end_address, PHYSTOP)` 范围内。
3. `walk` 无限崩溃：最可能是新分配的页表没有清零，其中一个随机的 `PTE_V=1` 的脏条目对应的地址根本不可用。记住：`kalloc()` 返回的页必须 `memset(page, 0, PGSIZE)` 清零！

## 进阶实验验收题

1. 观察 `walk` 函数：当 `alloc=0` 且目标 PTE 不存在时返回 0。请问在真实内核中，什么情况下调用 `walk` 时会故意传 `alloc=0`？（提示：考虑“查询”和“建立”的语义差异）
  2. 在 `kvminithart()` 中，调用 `sfence_vma()` 的目的是刷新 TLB。如果不调用它，即使页表设置正确，CPU 可能仍然用旧地址翻译继续执行，导致访问非法地址。请解释 TLB 的工作原理，以及为什么切换页表后必须刷新它。
- 

## ★ 本实验完成后的全貌

Makefile SRCS 状态：`entry.S + uart.c + main.c + console.c + kalloc.c + vm.c`  
`start_main()` 当前形态：

```
1 void start_main() {
2     kinit(); kvmininit(); kvminithart();
3     clear_screen();
4     printf("Paging enabled!\n");
5     while(1);
6 }
```

Lab4 预告：下一个实验新增 `start.c`（M-Mode降权）、`kernelvec.S`（陷阱入口汇编）和 `trap.c`（中断分发），届时每隔约0.1秒屏幕打印一次 `Tick!`。